

TARADOMEMUSIK

SYSTEM REQUIREMENTS DOCUMENT (SRD)

Version 1.0

Prepared For: Engineering & Architecture Team

Prepared By: Taradome Entertainment Group

1. EXECUTIVE OVERVIEW

TaradomeMusik is a **tech-driven music and entertainment platform** designed to integrate:

- Artist Development
- CRM-Driven Fan Monetization
- Music Distribution
- Sync Licensing
- Touring & Events
- Digital Monetization (VR/AR/NFT/Metaverse)
- Accounting & Royalty Systems
- Artist Microsites & Landing Pages

This platform supports the company's strategic objectives described in:

- Mission & Vision Framework

TaradomeMusik - \$2.5M Fundraise

- CRM Monetization Model

TaradomeMusik Mordern Musiic Re...

- Chart of Accounts Framework

Accounting Framework TaradomeMu...

- Website Architecture Blueprint

Web Design Flow for Taradome

- Founder Vision & Core Strategy

Taradome History and vision

2. SYSTEM ARCHITECTURE OVERVIEW

2.1 Architecture Type

Microservices-based architecture with:

Frontend:

- Next.js (App Router)
- TailwindCSS
- TypeScript

Backend:

- Node.js (Express or NestJS)
- REST + GraphQL APIs
- Webhooks for CRM integrations

Database:

- PostgreSQL (core data)
- Redis (cache/session)
- S3-compatible object storage (audio/media)

Infrastructure:

- AWS / Azure / GCP
 - CDN (Cloudflare)
 - CI/CD pipeline (GitHub Actions)
-

3. CORE SYSTEM MODULES

3.1 WEBSITE FRONTEND

Public Pages

- Homepage
- Artists
- Artist Microsites
- Music
- Events
- Spotlight (Artist of the Month)
- About
- Blog/Press
- Contact

Premium Sections

- Fan Dashboard
- Subscription Portal
- NFT / Digital Collectibles Portal
- Virtual Event Access
- Merch Store
- Investor Portal (restricted)

4. FUNCTIONAL REQUIREMENTS

4.1 ARTIST MANAGEMENT SYSTEM

Inputs:

- Artist bio
- Discography metadata
- Track stems (naming standard must follow:

TaradomeMusik Track Nomenclature

)

- Social links
- Contract type
- Royalty splits

Outputs:

- Artist microsite
 - Streaming embeds
 - Tour calendar
 - Revenue dashboard (internal only)
 - Fan segmentation data
-

4.2 MUSIC RELEASE MODULE

Inputs:

- Audio file (.WAV)
- Cover art
- ISRC code
- Release date
- Rights metadata

Outputs:

- 60-second preview stream
 - DSP redirection
 - CRM trigger events
 - Streaming analytics dashboard
-

4.3 CRM INTEGRATION (Modern Music Partnership Model)

Based on revenue strategy document

TaradomeMusik Mordern Musiic Re...

Required Capabilities:

- Fan segmentation
- Geolocation targeting
- Subscription tier automation
- Campaign ROI tagging
- Street team referral tracking

Required CRM API Features:

- POST /fan
 - GET /fan/{id}
 - POST /campaign
 - POST /segment
 - Webhook: purchase_event
 - Webhook: subscription_event
-

4.4 SUBSCRIPTION ENGINE

Tiered Model:

Bronze – Free

Silver – \$3–5/month

Gold – \$10–15/month

Inputs:

- Payment token
- Tier selection
- User profile data

Outputs:

- Recurring billing
- CRM tagging
- Exclusive content unlock
- Loyalty scoring

Payment APIs:

- Stripe
 - PayPal
 - Flutterwave (Africa)
-

4.5 MERCH & ECOMMERCE

Inputs:

- SKU
- Inventory count
- Artist tag

- QR referral code

Outputs:

- Order record
 - Revenue ledger entry
 - CRM behavior trigger
-

4.6 ACCOUNTING & LEDGER INTEGRATION

Based on Chart of Accounts

Accounting Framework TaradomeMu...

API Requirements:

POST /ledger/entry

GET /ledger/report?department=

GET /royalty/artist/{id}

Ledger must support:

- Account codes (XXX-XX format)
 - Artist project suffix
 - Department tracking
 - Royalty payable tracking
-

4.7 ROYALTY SYSTEM

Inputs:

- Streaming reports
- Sync fees
- Merch sales
- Tour revenue

Outputs:

- Artist royalty statement
- Recoupment tracking
- Payable calculation

4.8 SPOTLIGHT PAGE (Artist of the Month)

Inputs:

- Artist ID
- Featured track
- Video embed
- Campaign narrative

Outputs:

- Homepage hero placement
 - CRM spotlight tag
 - Analytics tracking
-

5. PREMIUM SITE REQUIREMENTS

5.1 NFT / Blockchain Layer

Optional but required for scalability:

- Mint via Zora / OpenSea
 - Smart contract royalties
 - Wallet connect integration
-

5.2 VR / AR EXPERIENCE

As defined in digital strategy

TaradomeMusik - \$2.5M Fundraise

Must support:

- Ticketed virtual concerts
- Avatar integration

- NFT gated events
-

5.3 METaverse LAND INTEGRATION

- Web3 login
 - NFT access control
 - Crypto payment integration
-

6. DATA MODEL REQUIREMENTS

Core Entities:

- User
 - Artist
 - Album
 - Track
 - Subscription
 - Transaction
 - LedgerEntry
 - RoyaltyStatement
 - Campaign
 - Event
-

7. SECURITY REQUIREMENTS

- GDPR compliance
 - CCPA compliance
 - AES-256 encryption (at rest)
 - HTTPS TLS 1.3
 - OAuth2 authentication
 - Role-based access control
-

8. PERFORMANCE REQUIREMENTS

- Page load under 2.5s

- Audio stream latency under 500ms
 - 99.9% uptime SLA
 - CDN caching enabled
-

9. ANALYTICS & KPIs

Track:

- CRM growth rate
 - Subscription conversion rate
 - Stream-to-sub conversion
 - Merch conversion
 - Tour geo heatmaps
 - Street team ROI
-

10. FUTURE UPGRADES

Phase 2:

- AI recommendation engine
- AI-generated remix engine
- Digital twin avatars

Phase 3:

- Proprietary streaming platform
 - Blockchain royalty automation
 - Smart contract distribution
-

11. DEPLOYMENT ROADMAP

Phase 1 – Core Website + CRM

Phase 2 – Subscription + Merch

Phase 3 – Ledger + Royalty Automation

Phase 4 – NFT + Metaverse

12. CONCLUSION

This system is designed to:

- Support TaradomeMusik's tech-driven vision
TaradomeMusik - \$2.5M Fundraise
- Enable CRM-powered monetization
TaradomeMusik Mordern Musiic Re...
- Integrate accounting scalability
Accounting Framework TaradomeMu...
- Deliver artist-first infrastructure
Taradome History and vision

TARADOMEMUSIK

SYSTEM REQUIREMENTS DOCUMENT (SRD)

Version 2.0

Technical Specification for Engineering Teams

1. SYSTEM OVERVIEW

TaradomeMusik is a technology-driven music and entertainment platform integrating:

- Artist development
- CRM-driven monetization
- Subscription & membership tiers
- Sponsorship & partnership programs
- Accounting ledger automation

- Royalty management
- NFT / VR / AR premium experiences
- Artist microsites
- Advanced analytics

The system must be modular, API-first, and scalable across Africa and global markets.

2. SYSTEM ARCHITECTURE

Frontend

- Next.js (App Router)
- TypeScript
- TailwindCSS
- Progressive Web App support

Backend

- Node.js (NestJS preferred)
- REST + GraphQL APIs
- Event-driven architecture
- Webhook processing layer

Database

- PostgreSQL (relational core)
- Redis (sessions/cache)
- S3-compatible object storage (audio/video)

Infrastructure

- AWS / Azure / GCP
- Cloudflare CDN
- Dockerized deployment
- CI/CD via GitHub Actions

Security

- OAuth2 + JWT
- TLS 1.3
- AES-256 encryption at rest
- Role-based access control (RBAC)
- GDPR & CCPA compliance

3. CORE SYSTEM MODULES

1. Public Website
 2. Artist Landing Pages
 3. CRM Engine
 4. Subscription Engine
 5. Merch & E-Commerce
 6. Accounting & Ledger System
 7. Royalty Engine
 8. Partnership & Sponsorship Portal
 9. Analytics & Reporting Dashboard
 10. Premium Digital Experience Layer
-

4. ARTIST LANDING PAGE REQUIREMENTS

Inputs

- Artist bio
- Discography metadata
- ISRC codes
- Streaming platform IDs
- Social links
- Royalty splits
- Tour dates
- Media assets

Outputs

- Public microsite
- Embedded streaming
- 60-second preview streaming
- Tour calendar
- CRM tagging events
- Revenue dashboard (internal)
- Sponsorship exposure tracking

APIs

GET /artist/{id}
POST /artist
PUT /artist/{id}
GET /artist/{id}/analytics
GET /artist/{id}/royalties

5. CRM SYSTEM REQUIREMENTS

Inputs

- Fan registration data
- Purchase history
- Subscription tier
- Geolocation
- Engagement metrics
- Referral source

Outputs

- Segmented fan lists
- Automated email/SMS triggers
- Geo-targeted campaign routing
- Campaign ROI tracking
- Street-team tracking

APIs

POST /crm/fan
GET /crm/fan/{id}
POST /crm/segment
POST /crm/campaign
Webhook: purchase_event
Webhook: subscription_event

6. ACCOUNTING LEDGER REQUIREMENTS

Must follow structured chart of accounts model.

Inputs

- Streaming revenue events
- Merch sales
- Tour revenue
- Sync licensing fees
- Marketing spend
- A&R spend
- Admin expenses

Outputs

- Departmental P&L
- Artist project profitability
- Royalty payable statements
- Recoupment tracking
- Sponsor revenue allocation

APIs

POST /ledger/entry

GET /ledger/report?department=

GET /ledger/project/{id}

POST /royalty/calculate

GET /royalty/artist/{id}

Ledger must support:

- Structured account numbers (XXX-XX)
- Artist/project suffix codes
- Department tagging
- Real-time posting

7. PARTNERSHIP & SPONSORSHIP PORTAL

Inputs

- Brand partner data
- Sponsorship tier definitions

- Campaign budgets
- Deliverables
- Media exposure data

Outputs

- Public sponsorship page
- Partner dashboard (restricted)
- Revenue tracking by sponsor
- Exposure analytics
- CRM cross-segmentation

APIs

POST /partner

GET /partner/{id}

POST /sponsorship/campaign

GET /sponsorship/report

GET /sponsorship/roi

Must integrate:

- Accounting ledger
- CRM segmentation
- Artist microsites

8. SUBSCRIPTION & PREMIUM FEATURES

Membership Tiers

Bronze – Free

Silver – \$3–5/month

Gold – \$10–15/month

Premium Requirements

- Gated content access
- NFT ownership validation
- VR event ticket validation

- Early music access
- Loyalty scoring
- Limited-edition merch unlock
- Sponsor-exclusive access

Payment APIs

Stripe
PayPal
Flutterwave

9. PERFORMANCE REQUIREMENTS

- Page load under 2.5 seconds
 - Audio preview latency under 500ms
 - 99.9% uptime SLA
 - Global CDN deployment
 - Horizontal scalability
 - API response under 300ms (95th percentile)
-

10. ANALYTICS REQUIREMENTS

Must track:

- CRM growth rate
 - Subscription conversion rate
 - Stream-to-subscription conversion
 - Merch conversion rate
 - Geo-tour heatmaps
 - Sponsor ROI
 - Artist revenue contribution
-

11. FUTURE UPGRADES

Phase 2

- AI fan recommendation engine
- Predictive tour routing
- Automated sponsor matching

Phase 3

- NFT minting automation
- Smart contract royalty splits
- Blockchain ledger sync

Phase 4

- Proprietary streaming platform
- AI-generated remix engine
- Digital twin avatars

1) Complete technical database schema (ERD)

Core entity model (PostgreSQL)

Conventions

- Primary keys: `uuid` (default `gen_random_uuid()`)
- Timestamps: `created_at`, `updated_at`
- Soft delete: `deleted_at` nullable where needed
- Multitenant-ready: `org_id` on most tables (optional if single tenant)

Tables (key fields)

orgs

- `id` (`uuid`, `pk`)
- `name` (`text`)
- `slug` (`text`, `unique`)
- `settings` (`jsonb`)

users

- `id` (`uuid`, `pk`)
- `org_id` (`uuid`, `fk` -> `orgs.id`)
- `email` (`citext`, `unique`)
- `phone` (`text`)
- `password_hash` (`text`) (*or external IdP*)
- `role` (`enum`: `admin`, `staff`, `artist_manager`, `finance`, `partner`, `viewer`)
- `status` (`enum`: `active`, `disabled`)

- last_login_at (timestampz)

artists

- id (uuid, pk)
- org_id (uuid, fk)
- name (text)
- slug (text, unique per org)
- bio (text)
- origin (text)
- genre (text)
- image_url (text)
- socials (jsonb) (*ig, tiktok, x, youtube, etc*)
- status (enum: active, paused, archived)

artist_members

(artist team access)

- id (uuid, pk)
- org_id
- artist_id (fk -> artists.id)
- user_id (fk -> users.id)
- role (enum: artist, manager, agent, editor, viewer)

releases

- id (uuid, pk)
- org_id
- artist_id (fk)
- title (text)
- type (enum: single, ep, album)
- release_date (date)
- cover_url (text)
- label_code (text) (*optional*)
- upc (text) (*optional*)
- status (enum: draft, scheduled, live, archived)

tracks

- id (uuid, pk)
- org_id
- release_id (fk -> releases.id)
- artist_id (fk -> artists.id) (*denormalized for speed*)
- title (text)
- isrc (text)
- duration_ms (int)
- explicit (bool)
- preview_asset_id (fk -> media_assets.id) (*60s preview*)

- audio_asset_id (fk -> media_assets.id) *(full master, restricted)*
- status (enum: draft, live, archived)

media_assets

- id (uuid, pk)
- org_id
- type (enum: image, audio, video, document)
- storage_provider (enum: s3, gcs, azure, local)
- bucket (text)
- object_key (text)
- mime_type (text)
- size_bytes (bigint)
- checksum_sha256 (text)
- metadata (jsonb) *(waveform, bitrate, width/height, etc)*

landing_pages

(artist landing pages + campaign pages + sponsor pages)

- id (uuid, pk)
- org_id
- type (enum: artist, release, spotlight, sponsor, partnership)
- artist_id (nullable fk)
- release_id (nullable fk)
- partner_id (nullable fk)
- slug (text)
- title (text)
- content (jsonb) *(blocks: hero, CTA, embeds, etc)*
- status (enum: draft, published, archived)
- published_at (timestampz)

spotlights

(Artist of the Month)

- id (uuid, pk)
 - org_id
 - month (date) *(store first day of month)*
 - artist_id (fk)
 - headline (text)
 - copy (text)
 - featured_track_id (nullable fk -> tracks.id)
 - cta_label (text)
 - cta_target (text) *(URL or internal route)*
 - status (enum: draft, published, archived)
-

CRM + fan monetization

fans

- id (uuid, pk)
- org_id
- email (citext, indexed)
- phone (text, indexed)
- first_name (text)
- last_name (text)
- country (text)
- city (text)
- region (text)
- source (text) (*utm/source/landing page*)
- consent_email (bool)
- consent_sms (bool)
- consent_ts (timestampz)
- traits (jsonb) (*flex attributes*)
- Unique constraints (per org): (email) optional, (phone) optional

fan_tags

- id (uuid, pk)
- org_id
- name (text) (*e.g., spotlight, artist:irhay, atlanta, buyer*)
- type (enum: system, user)

fan_tag_map

- fan_id (fk -> fans.id)
- tag_id (fk -> fan_tags.id)
- created_at

campaigns

- id (uuid, pk)
- org_id
- name (text)
- channel (enum: email, sms, push, ads, social)
- status (enum: draft, active, paused, ended)
- budget_cents (bigint)
- starts_at, ends_at
- metadata (jsonb)

events

(tracking + attribution)

- id (uuid, pk)
 - org_id
 - fan_id (nullable fk)
 - type (text) (*page_view, signup, preview_play, purchase, subscribe, etc*)
 - entity_type (text) (*artist/release/track/partner*)
 - entity_id (uuid)
 - utm (jsonb)
 - ip (inet)
 - user_agent (text)
 - occurred_at (timestampz)
-

Subscriptions + payments

subscription_plans

- id (uuid, pk)
- org_id
- name (text) (*Bronze/Silver/Gold*)
- price_cents (int)
- interval (enum: month, year)
- features (jsonb)
- provider_product_id (text) (*Stripe product/price ids*)

subscriptions

- id (uuid, pk)
- org_id
- fan_id (fk -> fans.id)
- plan_id (fk -> subscription_plans.id)
- status (enum: active, past_due, canceled, paused)
- provider (enum: stripe, paypal, flutterwave)
- provider_subscription_id (text)
- started_at, ended_at, cancel_at

payments

- id (uuid, pk)
- org_id
- fan_id (nullable fk)
- subscription_id (nullable fk)
- amount_cents (bigint)
- currency (text)
- provider (enum)
- provider_payment_id (text)
- status (enum: succeeded, failed, pending, refunded)
- paid_at (timestampz)

- metadata (jsonb)
-

Partnerships + sponsorships

partners

- id (uuid, pk)
- org_id
- name (text)
- type (enum: sponsor, brand, distributor, media, venue)
- contact_name (text)
- contact_email (citext)
- contact_phone (text)
- notes (text)
- status (enum: active, inactive)

sponsorship_packages

- id (uuid, pk)
- org_id
- name (text) (*Gold Sponsor, Title Sponsor, etc*)
- price_cents (bigint)
- deliverables (jsonb) (*impressions, logo placements, content, etc*)

sponsorship_campaigns

- id (uuid, pk)
- org_id
- partner_id (fk)
- package_id (fk)
- name (text)
- start_date, end_date
- budget_cents (bigint)
- status (enum: draft, active, complete)
- kpis (jsonb)

deliverables

- id (uuid, pk)
- org_id
- campaign_id (fk)
- type (enum: logo_placement, social_post, video_integration, event_booth, email_blast)
- due_at (timestampz)
- status (enum: pending, delivered, approved)
- evidence_urls (jsonb)

Accounting ledger + royalties

chart_of_accounts

- id (uuid, pk)
- org_id
- code (text) (*e.g., 400-10*)
- name (text)
- type (enum: asset, liability, equity, revenue, expense)
- parent_id (nullable fk -> chart_of_accounts.id)
- active (bool)

ledger_entries

- id (uuid, pk)
- org_id
- posted_at (timestampz)
- source_type (text) (*payment, distro_report, invoice, manual*)
- source_id (uuid)
- memo (text)
- currency (text)
- status (enum: pending, posted, reversed)

ledger_lines

- id (uuid, pk)
- org_id
- entry_id (fk -> ledger_entries.id)
- account_id (fk -> chart_of_accounts.id)
- artist_id (nullable fk -> artists.id)
- partner_id (nullable fk -> partners.id)
- department (text) (*A&R, Marketing, Admin, Touring, etc*)
- debit_cents (bigint)
- credit_cents (bigint)
- metadata (jsonb) (*project tags, invoice refs, etc*)

royalty_profiles

- id (uuid, pk)
- org_id
- artist_id (fk)
- default_rate_bps (int) (*basis points*)
- recoupable_balance_cents (bigint)
- payment_terms (jsonb)

royalty_statements

- id (uuid, pk)
- org_id
- artist_id (fk)
- period_start, period_end
- gross_cents, net_cents
- recouped_cents
- payable_cents
- status (enum: draft, issued, paid)

royalty_lines

- id (uuid, pk)
- org_id
- statement_id (fk)
- track_id (nullable fk)
- revenue_type (enum: streaming, sync, merch, tour, sponsor)
- units (numeric)
- gross_cents, net_cents
- rate_bps (int)
- artist_share_cents

Mermaid ERD (engineers can paste into docs)

2) Full API spec (OpenAPI / Swagger)

OpenAPI 3.0 YAML (baseline)

Paste into Swagger Editor or your repo as openapi.yaml.

```
openapi: 3.0.3
info:
  title: TaradomeMusik Platform API
  version: 1.0.0
  description: API-first platform for TaradomeMusik website, CRM, accounting ledger, sponsorships, and artist landing pages.
servers:
  - url: https://api.taradomemusik.com/v1
security:
  - bearerAuth: []
tags:
  - name: Auth
  - name: Artists
  - name: Releases
```


- name: Tracks
- name: Media
- name: LandingPages
- name: Spotlight
- name: CRM
- name: Subscriptions
- name: Payments
- name: Partners
- name: Sponsorships
- name: Ledger
- name: Royalties
- name: Webhooks

components:

securitySchemes:
 bearerAuth:
 type: http
 scheme: bearer
 bearerFormat: JWT

schemas:

Error:
 type: object
 properties:
 code: { type: string }
 message: { type: string }
 details: { type: object, additionalProperties: true }

Artist:

 type: object
 properties:
 id: { type: string, format: uuid }
 name: { type: string }
 slug: { type: string }
 bio: { type: string }
 origin: { type: string }
 genre: { type: string }
 imageUrl: { type: string }
 socials: { type: object, additionalProperties: true }
 status: { type: string, enum: [active, paused, archived] }

Release:

 type: object
 properties:
 id: { type: string, format: uuid }
 artistId: { type: string, format: uuid }
 title: { type: string }
 type: { type: string, enum: [single, ep, album] }
 releaseDate: { type: string, format: date }
 coverUrl: { type: string }
 status: { type: string, enum: [draft, scheduled, live, archived] }

Track:

 type: object
 properties:
 id: { type: string, format: uuid }
 releaseId: { type: string, format: uuid }

```
  artistId: { type: string, format: uuid }
  title: { type: string }
  isrc: { type: string }
  durationMs: { type: integer }
  previewUrl: { type: string }
  status: { type: string, enum: [draft, live, archived] }
```

Fan:

```
  type: object
  properties:
    id: { type: string, format: uuid }
    email: { type: string, format: email }
    phone: { type: string }
    firstName: { type: string }
    lastName: { type: string }
    city: { type: string }
    region: { type: string }
    country: { type: string }
    consentEmail: { type: boolean }
    consentSms: { type: boolean }
    tags: { type: array, items: { type: string } }
```

LedgerEntryCreate:

```
  type: object
  required: [postedAt, currency, lines]
  properties:
    postedAt: { type: string, format: date-time }
    memo: { type: string }
    currency: { type: string, example: USD }
    sourceType: { type: string }
    sourceId: { type: string, format: uuid }
    lines:
      type: array
      minItems: 2
      items:
        type: object
        required: [accountCode]
        properties:
          accountCode: { type: string, example: "400-10" }
          debitCents: { type: integer, example: 0 }
          creditCents: { type: integer, example: 10000 }
          artistId: { type: string, format: uuid }
          partnerId: { type: string, format: uuid }
          department: { type: string }
          metadata: { type: object, additionalProperties: true }
```

paths:

```
  /auth/login:
    post:
      tags: [Auth]
      security: []
      summary: Login and obtain JWT
      requestBody:
        required: true
        content:
          application/json:
            schema:
```

```

        type: object
        required: [email, password]
        properties:
            email: { type: string, format: email }
            password: { type: string, format: password }
responses:
  "200":
    description: JWT returned
    content:
      application/json:
        schema:
          type: object
          properties:
            token: { type: string }
  "401":
    description: Unauthorized
    content:
      application/json:
        schema: { $ref: "#/components/schemas/Error" }

/artists:
  get:
    tags: [Artists]
    summary: List artists
    responses:
      "200":
        description: Array of artists
        content:
          application/json:
            schema:
              type: array
              items: { $ref: "#/components/schemas/Artist" }
  post:
    tags: [Artists]
    summary: Create artist
    requestBody:
      required: true
      content:
        application/json:
          schema: { $ref: "#/components/schemas/Artist" }
    responses:
      "201":
        description: Created
        content:
          application/json:
            schema: { $ref: "#/components/schemas/Artist" }

/artists/{artistId}:
  get:
    tags: [Artists]
    summary: Get artist
    parameters:
      - in: path
        name: artistId
        required: true
        schema: { type: string, format: uuid }
    responses:

```

```

    "200":
      description: Artist
      content:
        application/json:
          schema: { $ref: "#/components/schemas/Artist" }

/releases:
  get:
    tags: [Releases]
    summary: List releases
    responses:
      "200":
        description: Releases
        content:
          application/json:
            schema:
              type: array
              items: { $ref: "#/components/schemas/Release" }

/releases/{releaseId}/tracks:
  get:
    tags: [Tracks]
    summary: List tracks for a release
    parameters:
      - in: path
        name: releaseId
        required: true
        schema: { type: string, format: uuid }
    responses:
      "200":
        description: Tracks
        content:
          application/json:
            schema:
              type: array
              items: { $ref: "#/components/schemas/Track" }

/tracks/{trackId}/preview:
  get:
    tags: [Tracks]
    security: []
    summary: Get 60-second preview stream URL (public)
    parameters:
      - in: path
        name: trackId
        required: true
        schema: { type: string, format: uuid }
    responses:
      "302":
        description: Redirect to signed preview URL

/landing-pages:
  get:
    tags: [LandingPages]
    summary: List landing pages (admin)
    responses:
      "200":

```

```

        description: Pages
        content:
          application/json:
            schema:
              type: array
              items:
                type: object

/landing-pages/{slug}:
  get:
    tags: [LandingPages]
    security: []
    summary: Resolve published landing page by slug (public)
    parameters:
      - in: path
        name: slug
        required: true
        schema: { type: string }
    responses:
      "200":
        description: Page JSON (block model)
        content:
          application/json:
            schema:
              type: object

/spotlight/current:
  get:
    tags: [Spotlight]
    security: []
    summary: Get current artist spotlight (public)
    responses:
      "200":
        description: Spotlight payload
        content:
          application/json:
            schema:
              type: object

/crm/subscribe:
  post:
    tags: [CRM]
    security: []
    summary: Fan signup (captures consent + tags)
    requestBody:
      required: true
      content:
        application/json:
          schema:
            type: object
            required: [email, source]
            properties:
              email: { type: string, format: email }
              phone: { type: string }
              firstName: { type: string }
              lastName: { type: string }
              source: { type: string }

```

```

        tags: { type: array, items: { type: string } }
        consentEmail: { type: boolean, default: true }
        consentSms: { type: boolean, default: false }
responses:
  "200":
    description: Accepted
    content:
      application/json:
        schema:
          type: object
          properties:
            ok: { type: boolean }
            fanId: { type: string, format: uuid }

/partners:
  get:
    tags: [Partners]
    summary: List partners
    responses:
      "200":
        description: Partners
        content:
          application/json:
            schema:
              type: array
              items: { type: object }
  post:
    tags: [Partners]
    summary: Create partner
    requestBody:
      required: true
      content:
        application/json:
          schema: { type: object }
    responses:
      "201":
        description: Created

/sponsorships/campaigns:
  post:
    tags: [Sponsorships]
    summary: Create sponsorship campaign
    requestBody:
      required: true
      content:
        application/json:
          schema: { type: object }
    responses:
      "201":
        description: Created

/ledger/entries:
  post:
    tags: [Ledger]
    summary: Post a double-entry ledger entry
    requestBody:
      required: true

```

```

    content:
      application/json:
        schema: { $ref: "#/components/schemas/LedgerEntryCreate" }
  responses:
    "201":
      description: Posted
      content:
        application/json:
          schema:
            type: object
            properties:
              entryId: { type: string, format: uuid }
              status: { type: string }

/ledger/reports/trial-balance:
  get:
    tags: [Ledger]
    summary: Trial balance report
    parameters:
      - in: query
        name: from
        schema: { type: string, format: date }
      - in: query
        name: to
        schema: { type: string, format: date }
    responses:
      "200":
        description: Trial balance
        content:
          application/json:
            schema: { type: object }

/royalties/statements/generate:
  post:
    tags: [Royalties]
    summary: Generate royalty statements for a period
    requestBody:
      required: true
      content:
        application/json:
          schema:
            type: object
            required: [periodStart, periodEnd]
            properties:
              periodStart: { type: string, format: date }
              periodEnd: { type: string, format: date }
    responses:
      "202":
        description: Job accepted
        content:
          application/json:
            schema:
              type: object
              properties:
                jobId: { type: string }

/webhooks/stripe:

```

```
post:
  tags: [Webhooks]
  security: []
  summary: Stripe webhook receiver
  requestBody:
    required: true
    content:
      application/json:
        schema: { type: object }
  responses:
    "200":
      description: OK
```

3) DevOps deployment architecture diagram

Reference architecture (AWS example; maps cleanly to Azure/GCP)

Key properties

- Edge caching for site + media
- API behind WAF + rate limits
- Background jobs for royalties + reports
- Observability and audit logging for finance

CI/CD (GitHub Actions)

- PR → lint/test/build → preview deploy
- main → deploy web + api → run migrations → smoke tests → promote

Environments

- dev (feature branches)
 - staging (UAT + partner demos)
 - prod (restricted access controls)
-

4) CTO-level scaling blueprint memo (investor-ready)

Executive thesis

TaradomeMusik is designed as an **artist + fan monetization engine** where the website is the front door, **CRM is the growth brain**, and the **ledger/royalty stack is the trust engine**. The

system scales by making growth loops measurable (CRM + analytics) and making payouts auditable (ledger + statements).

Scaling priorities (0→1, 1→10, 10→100)

Phase A: 0→1 (Launch reliability)

Goals

- Fast site, frictionless signup, release previews, spotlight conversion
- Stable CRM ingestion + tagging
- Basic ledger posting from revenue events

Key investments

- Edge CDN, rate limiting, basic event pipeline
- Strict data contracts: Fan, Event, LedgerEntry, RoyaltyStatement
- Signed URLs for media; preview-only public access

KPIs

- Signup conversion, preview-to-click-through, CAC/CPA by campaign, uptime, API p95 latency

Phase B: 1→10 (Monetization + automation)

Goals

- Subscription tiers + gated content
- Sponsorship portal with deliverables tracking
- Automated royalty statement generation

Key investments

- Queue + worker fleet (royalties, reports, CRM automations)
- Idempotent webhooks (payments + CRM)
- Partition events by org/artist; build cohort analytics

KPIs

- MRR, churn, LTV, sponsor ROI, royalty cycle time (close-to-statement)

Phase C: 10→100 (Global scale + platformization)

Goals

- Multi-region expansion
- Self-serve artist microsites + embed widgets
- Near-real-time revenue dashboards per artist/partner

Key investments

- Read replicas + caching strategies
- Data warehouse (Snowflake/BigQuery) for analytics
- Modular “landing page builder” + API-driven microsite publishing

KPIs

- Cost per active fan, infra cost per 1k pageviews, processing throughput, data freshness SLA

Risk controls (investor-grade)

- **Financial integrity:** immutable ledger entries, audit trails, role-based approval for reversals
- **Data privacy:** consent model, suppression lists, regional compliance
- **Operational resilience:** backups + PITR, blue/green deploys, incident runbooks

Why this wins

Most music labels have “websites.” TaradomeMusik has a **growth + finance platform**: every page is a conversion surface, every campaign is measurable, and every payout is auditable—creating defensible trust with artists and scalable economics for investors.